

DLIC HW7

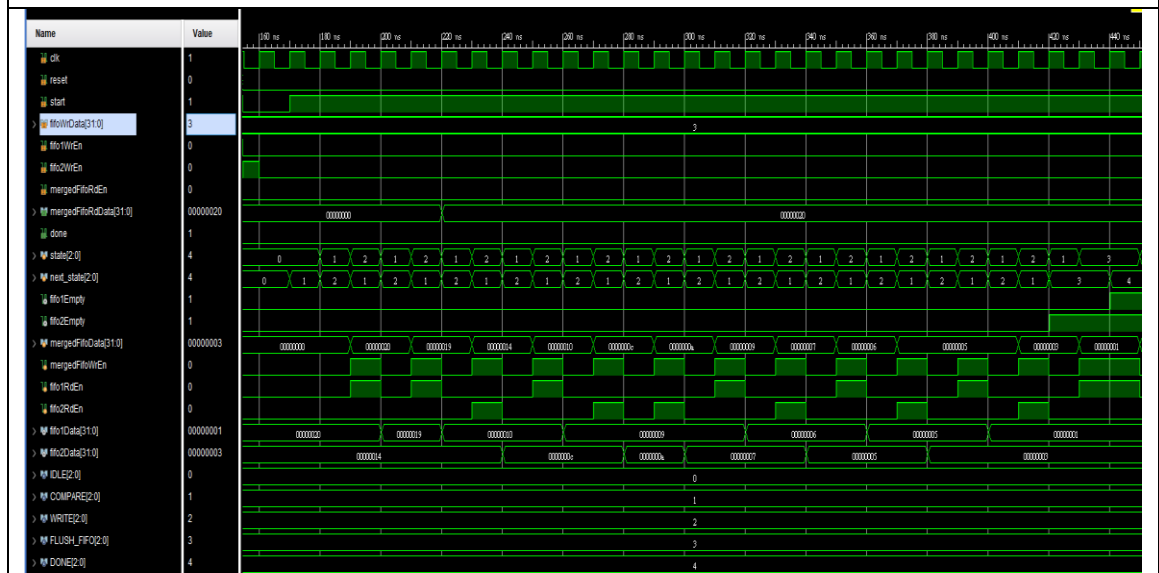
Designing a custom IP for Merge Operation with Xilinx FIFO Generator

系級:資訊系大四 學號:E34106010 姓名:黃偉峰

1. Homework Result

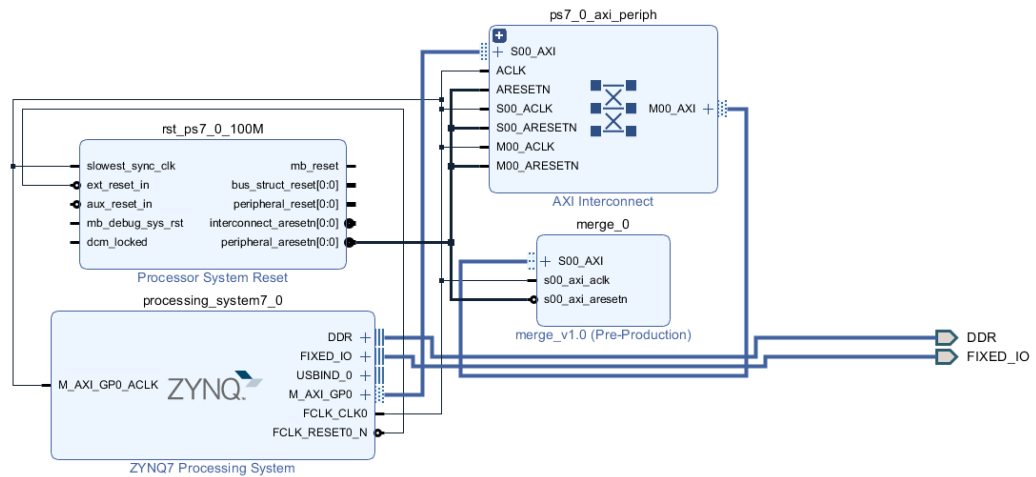


輸出波形圖



2. Code Explanation

Vivado Block Diagram



mergeCore HDL code

FSM:

```
always @(*) begin //next_state
    case(state)
        IDLE: next_state = (start) ? COMPARE : IDLE;
        COMPARE:begin
            if(!fifo1Empty && !fifo2Empty) next_state = WRITE;
            else next_state = FLUSH_FIFO;
        end
        WRITE: next_state = COMPARE;
        FLUSH_FIFO:begin
            if(fifo1Empty && fifo2Empty) next_state = DONE;
            else next_state = FLUSH_FIFO;
        end
        DONE: next_state = (start) ? DONE : IDLE;
        default: next_state = IDLE;
    endcase
end
```

IDLE: 等待 start 訊號開始 merge。

- 若 start 為 1 就代表 fifo1、fifo2 輸入完成開始做 merge，轉換到 COMPARE state。
- 若 start 為 0，則持續在 IDLE 等待。

COMPARE: 比較 fifo 當中最先進入的資料，兩者較大的寫出。

- 只要 fifo1、fifo2 當中一個不為空，就轉換到 WRITE state。
- 若 fifo1、fifo2 當中有一個為空，就轉換到 FLUSH_FIFO state，將另一個不為空的 FIFO 資料依序一路寫出。

WRITE: 將 COMPARE state 中拉高的寫出訊號或讀取訊號拉低

- 轉換到 COMPARE state。

FLUSH_FIFO: 將其中一個非空的 FIFO 資料全數讀出。

- 若 fifo1、fifo2 當中一個不為空，就待在 FLUSH_FIFO state。
- 若兩者皆為空代表 merge 完成，轉換到 DONE state。

DONE: merge 完成，將 done 訊號拉高。

- 若 start 持續為 1，則持續待在 DONE state。
- 若 start 訊號轉為 0，則轉換到 IDLE 等待下一次 merge 開始。

fifo1RdEn fifo2RdEn

```
always @(posedge clk) begin //fifo1RdEn fifo2RdEn
    if(reset)begin
        fifo1RdEn <= 1'b0;
        fifo2RdEn <= 1'b0;
    end else begin
        case(state)
            COMPARE:begin
                if(!fifo1Empty && !fifo2Empty)begin
                    if(fifo1Data > fifo2Data) fifo1RdEn <= 1'b1;
                    else fifo2RdEn <= 1'b1;
                end else if(fifo1Empty) fifo2RdEn <= 1'b1;
                else if(fifo2Empty) fifo1RdEn <= 1'b1;
            end
            WRITE:begin
                fifo1RdEn <= 1'b0;
                fifo2RdEn <= 1'b0;
            end
            FLUSH_FIFO:begin
                if(fifo1Empty && fifo2Empty)begin
                    fifo1RdEn <= 1'b0;
                    fifo2RdEn <= 1'b0;
                end else begin
                    if(fifo1Empty) fifo2RdEn <= 1'b1;
                    else if(fifo2Empty) fifo1RdEn <= 1'b1;
                end
            end
        endcase
    end
end
```

- COMPARE :
 - 比較兩 FIFO 中資料，將較大的那個 FIFO RdEn 拉高。
 - 若其中一個 FIFO 為空，就讀取另一個。
- WRITE : 關閉讀取（拉低 RdEn）。
- FLUSH_FIFO :
 - 若兩個 FIFO 都空，不讀。
 - 若有一個還有資料，就持續讀非空那個。

mergeFifoWrEn

```
always @(posedge clk) begin //mergeFifoWrEn
    if(reset) mergedFifoWrEn <= 1'b0;
    else begin
        case(state)
            COMPARE: mergedFifoWrEn <= 1'b1;
            WRITE: mergedFifoWrEn <= 1'b0;
            FLUSH_FIFO:begin
                if(!fifo1Empty || !fifo2Empty) mergedFifoWrEn <= 1'b1;
                else mergedFifoWrEn <= 1'b0;
            end
        endcase
    end
end
end
```

- COMPARE :
 - 比較完成後就開啟寫入到 mergedFifo (mergedFifoWrEn = 1)，將較大的資料送進 merged FIFO。
- WRITE :
 - 關閉寫入 (mergedFifoWrEn = 0)，避免重複寫入。
- FLUSH_FIFO :
 - 只要任一個 FIFO 還有資料，就繼續寫入 merged FIFO (mergedFifoWrEn = 1)。
 - 若兩個都空了，則關閉寫入。

mergedFifoData

```
always @(posedge clk) begin //mergedFifoData
    if(reset) mergedFifoData <= 32'b0;
    else begin
        case(state)
            COMPARE:begin
                if(!fifo1Empty && !fifo2Empty)begin
                    if(fifo1Data > fifo2Data) mergedFifoData <= fifo1Data;
                    else mergedFifoData <= fifo2Data;
                end else begin
                    if(fifo1Empty) mergedFifoData <= fifo2Data;
                    else mergedFifoData <= fifo1Data;
                end
            end
            FLUSH_FIFO:begin
                if(fifo1Empty) mergedFifoData <= fifo2Data;
                else mergedFifoData <= fifo1Data;
            end
        endcase
    end
end
```

- COMPARE 狀態：
 - 比較 fifo1Data 與 fifo2Data，取較大的那個作為合併資料。
 - 若其中一個 FIFO 為空，就選擇另一個的資料。
- FLUSH_FIFO 狀態：
 - 若其中一個 FIFO 還有資料，就把那筆資料寫進 mergedFifoData，直到兩個都空。

SDK code

```
#include "xparameters.h"
#include "merge.h"
#include "xil_io.h"

int main(){
    u32 a[] = {32,25,16,9,6,5,1};
    u32 b[] = {20,12,10,7,5,3};
    u32 c[13];
    u32 Status;
    for(int i=0;i<7;i++){
        MERGE_mWriteReg(XPAR_MERGE_0_S00_AXI_BASEADDR, 0xC, a[i]);
    }
    for(int i=0;i<6;i++){
        MERGE_mWriteReg(XPAR_MERGE_0_S00_AXI_BASEADDR, 0x10, b[i]);
    }

    MERGE_mWriteReg(XPAR_MERGE_0_S00_AXI_BASEADDR, 0x0, 0x1);

    Status = MERGE_mReadReg(XPAR_MERGE_0_S00_AXI_BASEADDR, 0x4);

    while(!Status)
        Status = MERGE_mReadReg(XPAR_MERGE_0_S00_AXI_BASEADDR, 0x4);

    for(int i=0;i<13;i++){
        c[i] =MERGE_mReadReg(XPAR_MERGE_0_S00_AXI_BASEADDR, 0x8);
        xil_printf("%d\n\r",c[i]);
    }

    return 0;
}
```

1. 初始化兩組資料：
 - a[] 和 b[] 為已排序的降序陣列。
2. 寫入 FIFO：
 - 將 a[] 寫入 fifo1 (offset 0xC)
 - 將 b[] 寫入 fifo2 (offset 0x10)
3. 啟動合併：
 - 將 start 寫入 slv_reg (offset 0x0)
4. 等待完成：
 - 持續讀取 done 訊號 (offset 0x4) 直到為 1
5. 讀出結果：
 - 從 merged FIFO (offset 0x8) 連續讀出 13 筆合併後的資料

3. Lesson Learn

透過本次實作，我學會了如何使用 Xilinx 提供的硬體驅動方式來控制自訂的 AXI-Lite IP。從資料寫入兩個 FIFO，到讀取合併後結果，整個過程讓我更熟悉了 AXI-Lite 接口的讀寫操作。在寫軟體時，我實際使用了 MERGE_mWriteReg 和 MERGE_mReadReg 這類巨集來取代助教所提供的版本，這兩個巨集為 xilinx IP 會自動生成的讀取寫入巨集，使用起來會更加直觀。

一開始接觸助教提供的 AXI-Lite wrapper 時，我對每個暫存器的位址與對應功能感到有些困惑。後來透過查閱 AXI-Lite 的運作邏輯跟 IP 的 Register Map，我才釐清了整體資料流程與控制機制。